

Git



GitHub



CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

08 — 부록

- GUI 프로그램 소개
- 고급 명령어 소개

09 — 설치와 회원가입

- git
- visual studio code
- github

CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

08 — 부록

- GUI 프로그램 소개
- 고급 명령어 소개

09 — 설치와 회원가입

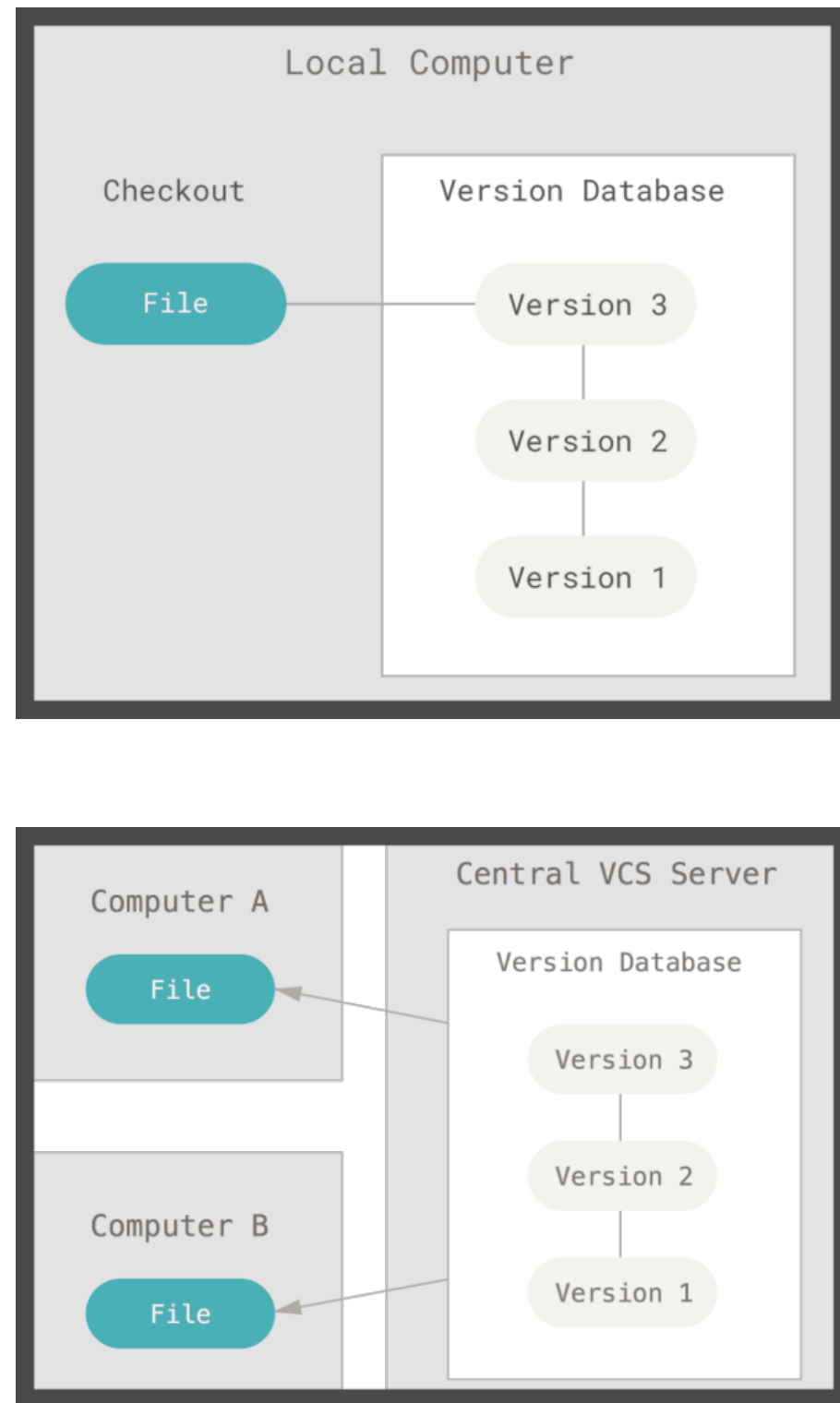
- git
- visual studio code
- github

01. Git이란?

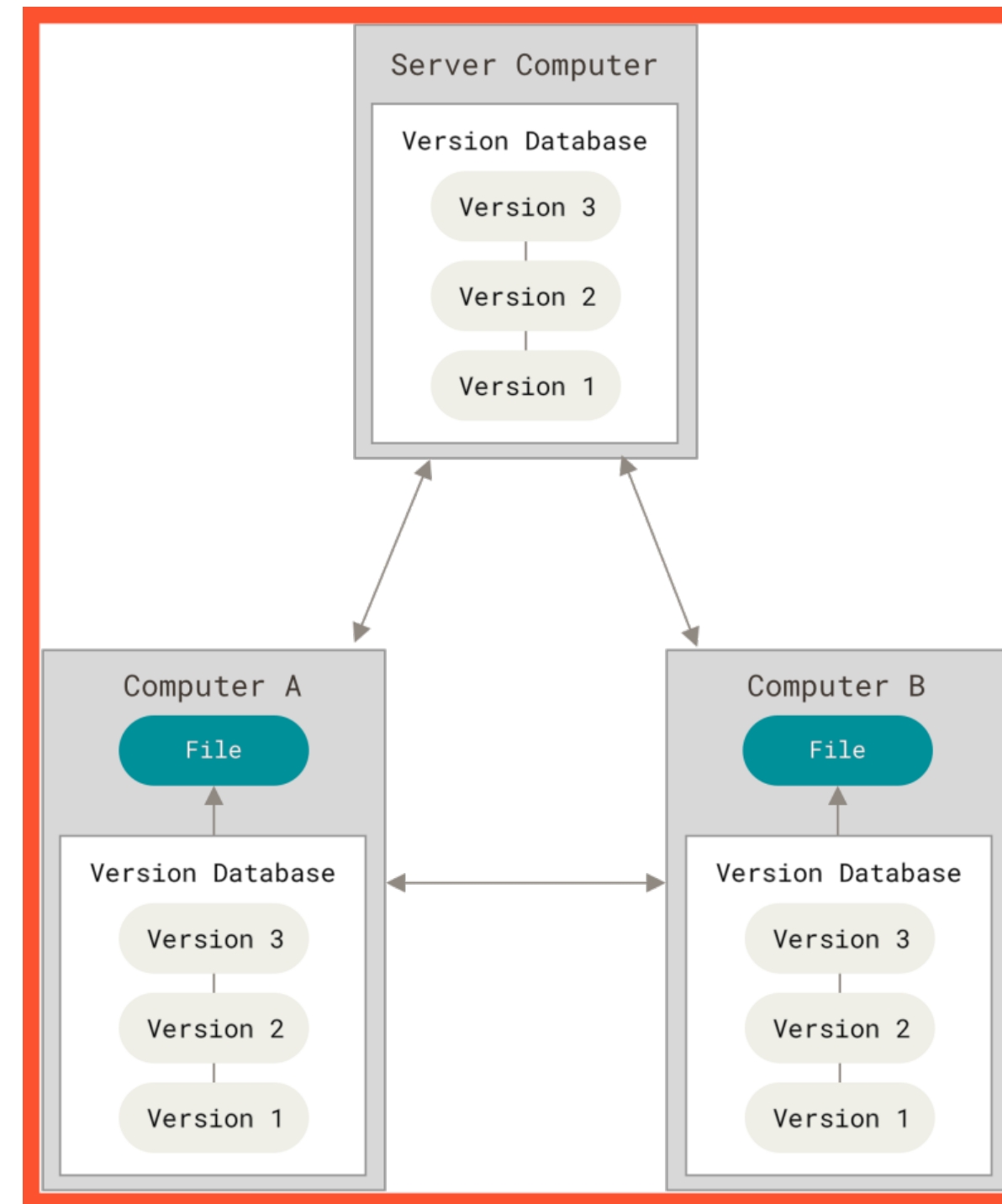
Git은 소스 코드의 변경 이력을 기록하고
여러 사람과 협업할 수 있도록 돕는
무료 오픈 소스 분산 버전 관리 시스템(DVCS)입니다.
파일의 수정 시점을 '스냅샷' 형태로 기록하여
언제든 이전 상태로 되돌릴 수 있습니다.

버전 관리 시스템은 파일 변화를 시간에 따라 기록했다가
나중에 특정 시점의 버전을 다시 꺼내올 수 있는 시스템이다.

01. Git이란?



분산 버전 관리 시스템(DVCS)



01. Git이란?

1. 협업 (여러명이 동시에)
2. 버전 관리 (세이프 포인트)



01. Git이란?

- 개발하기 위해서 Git을 쓰기 vs Git을 쓰기 위해서 개발하기
- 카톡/디스코드/이메일 vs Git/Github
- 작은 테스트용 vs 큰 프로젝트

- 결론 :



만약 비슷한 파일이 50개라면?

01. Git이란?

<https://www.linuxfoundation.org/blog/blog/10-years-of-git-an-interview-with-git-creator-linus-torvalds>

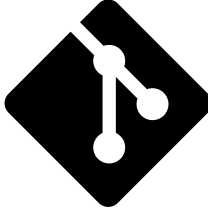
Why does Git work so well for Linux?

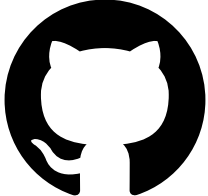
*Torvalds: Well, **it was obviously designed for our workflow**, so that is part of it. I've already mentioned the whole "distributed" part many times, but it bears repeating. But it was also designed to be efficient enough for a biggish project like Linux, and it was designed to do things that people considered "hard" before git – because those are the things *I* do every day.*

*Just to pick an example: the concept of "merging" was generally considered to be something really quite painful and hard in most SCM's. You'd plan your merges, because they were big deals. That's not acceptable to me, since I commonly do tens of merges a day when in the merge window, and even then, the biggest overhead shouldn't be the merge itself, it should be testing the result. **The "git" part of the merge is just a couple of seconds**, it should take me much longer just to write the merge explanation message.*

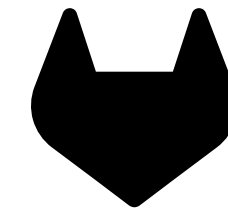
So git was basically designed and written for my requirements, and it shows.

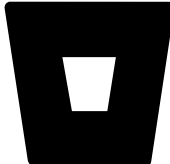
01. Git이란? - Git과 Github의 관계

 Git: 짜장면

 Github: 짜장면을 파는 유명 중국집1

Gitlab: 짜장면을 파는 유명 중국집2



 Bitbucket: 짜장면을 파는 유명 중국집3

CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

08 — 부록

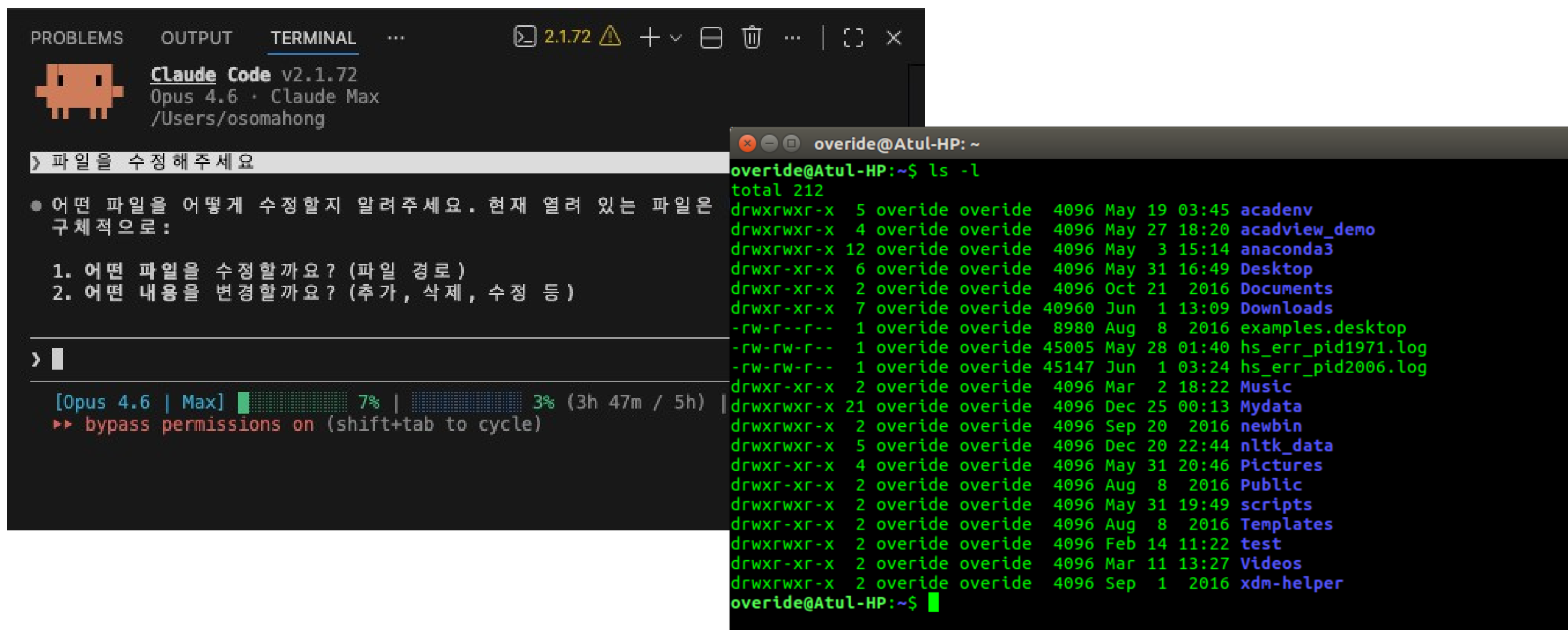
- GUI 프로그램 소개
- 고급 명령어 소개

09 — 설치와 회원가입

- git
- visual studio code
- github

02. 터미널과 CLI

검은배경에 글씨만 있는거 "CLI : Command Line Interface"



02. 터미널과 CLI

```
~$ git commit -m "hello"
```

```
$ codex "이 코드 설명해줘"
```

command [subcommand] [options] [arguments]

git

commit

-m

"hello"

codex

"코드를 설명해줘"

*주의: CLI에서는 구분을 '띄어쓰기'로 한다

*예시: `mkdir` 안녕하세요 저는 김성현입니다

1.안녕하세요 저는 김성현입니다

?

1.안녕하세요

2.저는

3.김성현입니다

02. 터미널과 CLI

- 대부분의 CLI 프로그램은 **상태**를 확인하는 명령어가 존재 -> 중요!
- 명령어 **단어에는 의미가 잘 들어있음** -> git switch 라면?
- 실행, 버그 상황에서 터미널에 **출력**되는 내용을 잘 읽어보세요.

개발자가 당신을 겁주기 위해서
이런 메시지를 적지는 않았을 것...

```
909696cb..d76764dd main -> origin/main
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
hint: preference for all repositories. You can also pass --rebase, --no-rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
```

02. 터미널과 CLI

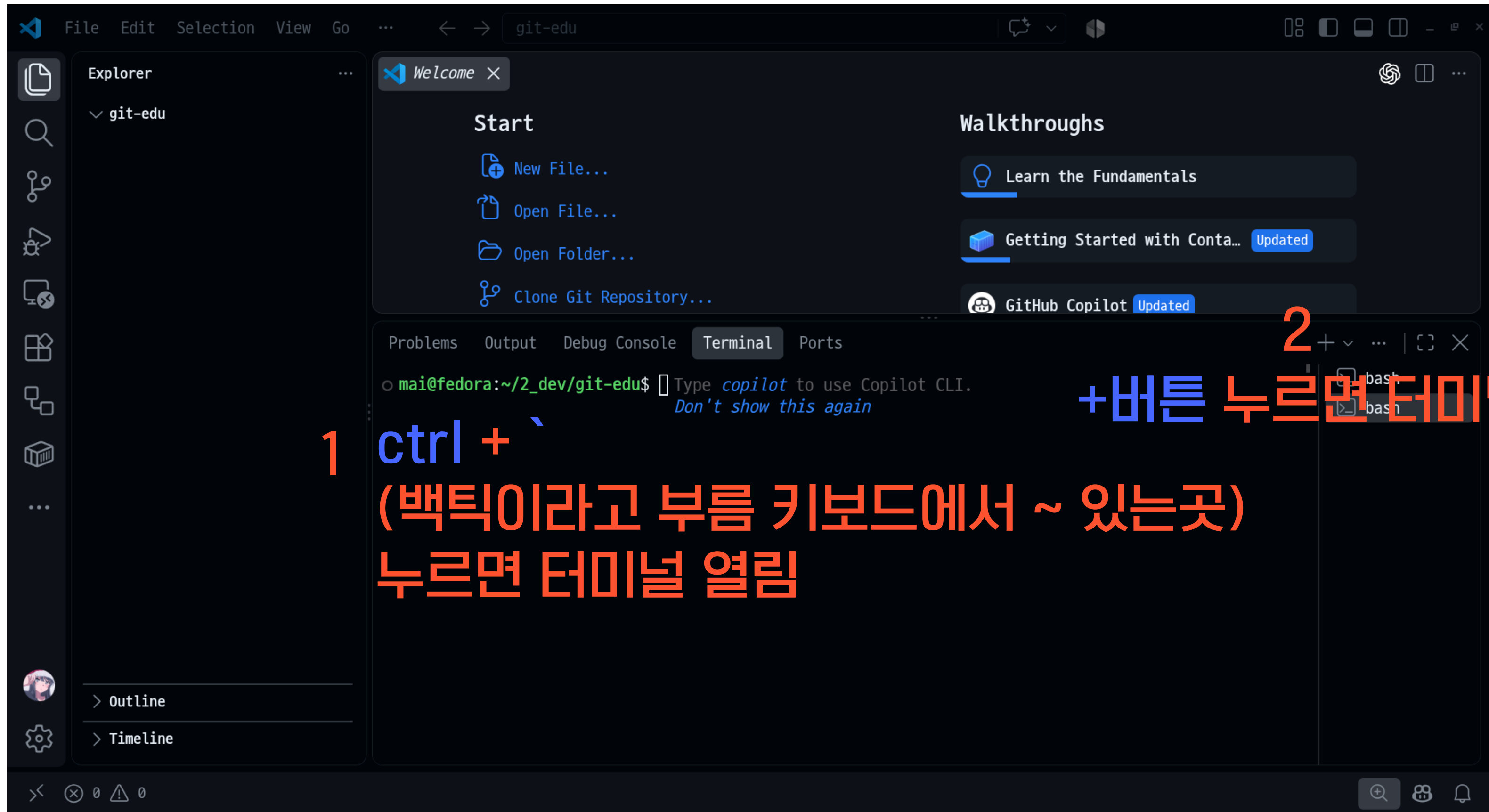
응, Git 입문 강의에서 “안전하게 확인하는 명령어들”로 소개하기엔 이 정도면 충분해.

- `git status` – 현재 상태
- `git log` – 커밋 기록
- `git diff` – 변경 내용
- `git branch` – 브랜치 목록
- `git remote -v` – 원격 저장소 정보
- `git config --list` – Git 설정
- `git --version` – Git 버전
- `git help <명령어>` – 도움말

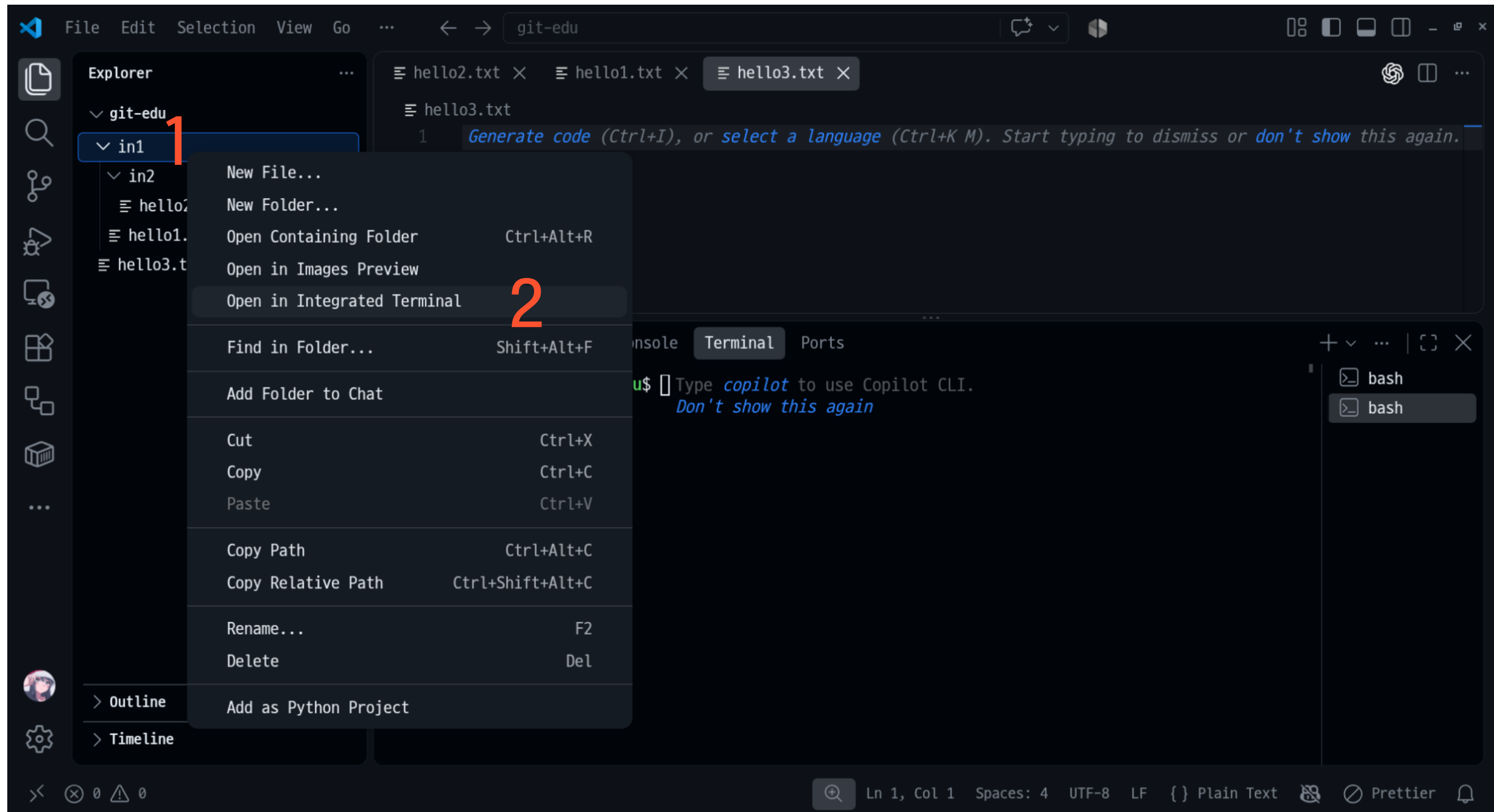
특히 초보자에게는 “이 명령어들은 확인용이니 부담 없이 써도 된다”고 묶어주면 좋아.

단, `git branch`는 인자 없이 쓸 때 조회용이라는 점만 살짝 덧붙이면 정확해.

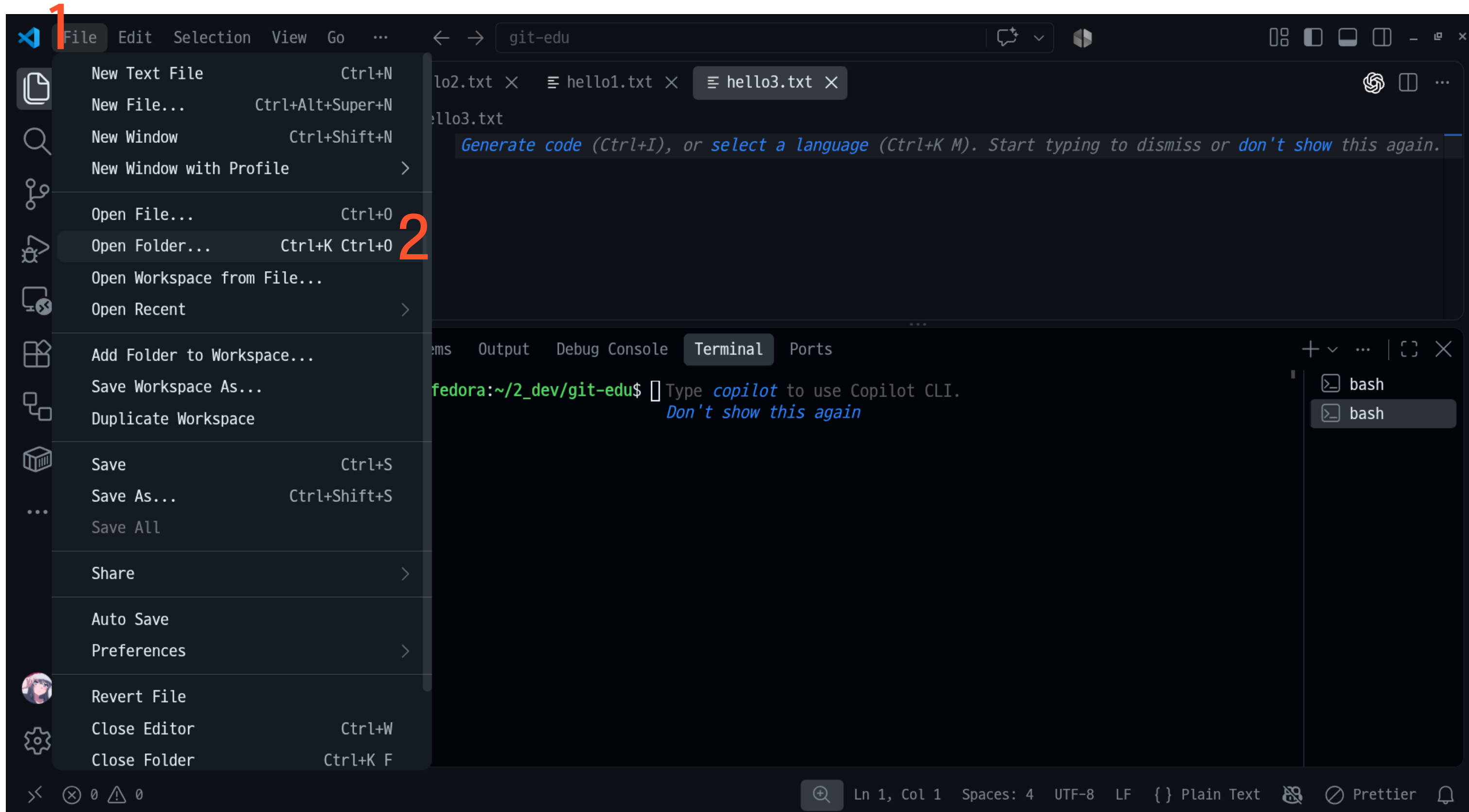
02. 터미널과 CLI



02. 터미널과 CLI



02. 터미널과 CLI



CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

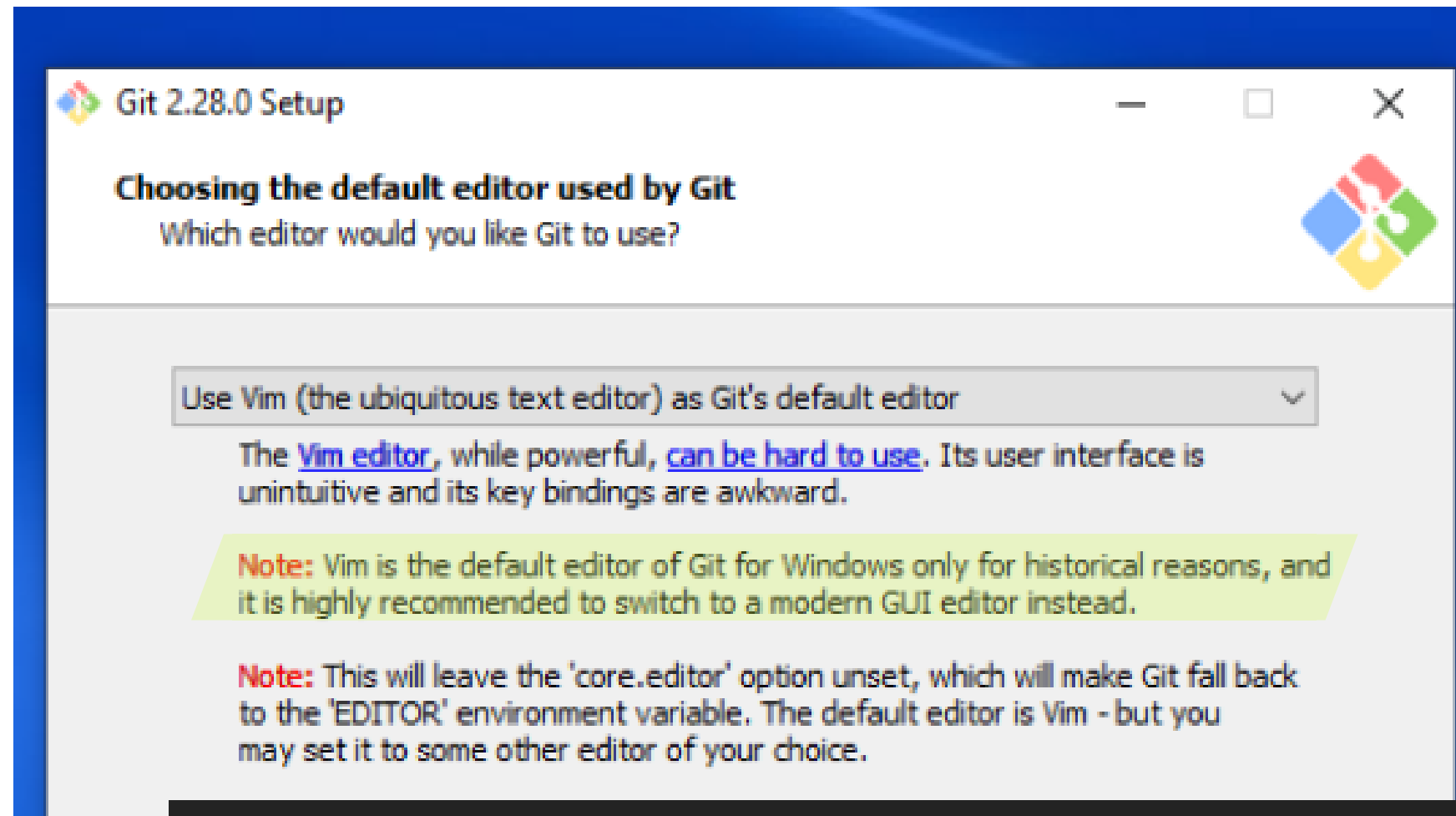
08 — 부록

- GUI 프로그램 소개
- 고급 명령어 소개

09 — 설치와 회원가입

- git
- visual studio code
- github

03. 초기 세팅



그냥 vscode 써라!

```
git config --global core.editor "code --wait"
```

03. 초기 세팅

```
git config --global --edit
```

```
[user]
name = Kimseonghyeon
email = 90964556+dev-
seonghyeon@users.noreply.github.com
[credential]
helper = store
[init]
defaultBranch = main
[core]
editor = code --wait
[pull]
rebase = false
[push]
autoSetupRemote = true
```

```
[user]
    name = Kimseonghyeon
    email = 90964556+dev-seon
[init]
    defaultBranch = main
[core]
    editor = code --wait
[pull]
    rebase = false
[push]
    autoSetupRemote = true
```

CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

08 — 부록

- GUI 프로그램 소개
- 고급 명령어 소개

09 — 설치와 회원가입

- git
- visual studio code
- github

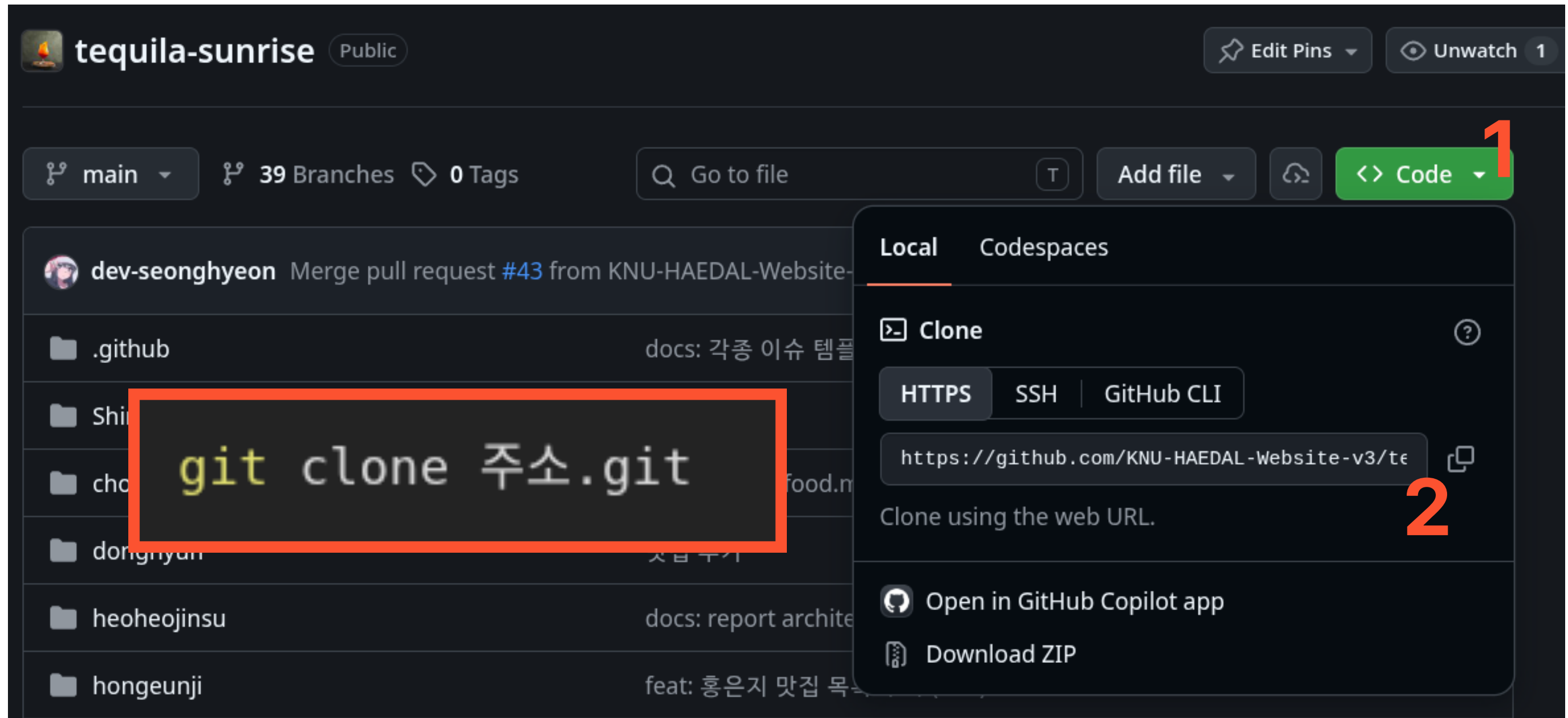
04. 레포지토리 초기 세팅

- 숨긴파일
- .git 폴더
- 직접 써넣는거 아님!!



04. 레포지토리 초기 세팅

git clone의 경우



04. 레포지토리 초기 세팅

git init의 경우

Quick setup — if you've done this kind of thing before

HTTPS

SSH

`https://github.com/dev-seonghyeon/- .git`

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# -" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/dev-seonghyeon/- .git
git push -u origin main
```

다 알려주잖아,,,,

...or push an existing repository from the command line

```
git remote add origin https://github.com/dev-seonghyeon/- .git
git branch -M main
git push -u origin main
```

```
echo "# 레포지토리 이름" >> README.md
git init
git add README.md
git commit -m "넣고 싶은 커밋 메시지"
git branch -M main
git remote add origin 주소.git
git push -u origin main
```

CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

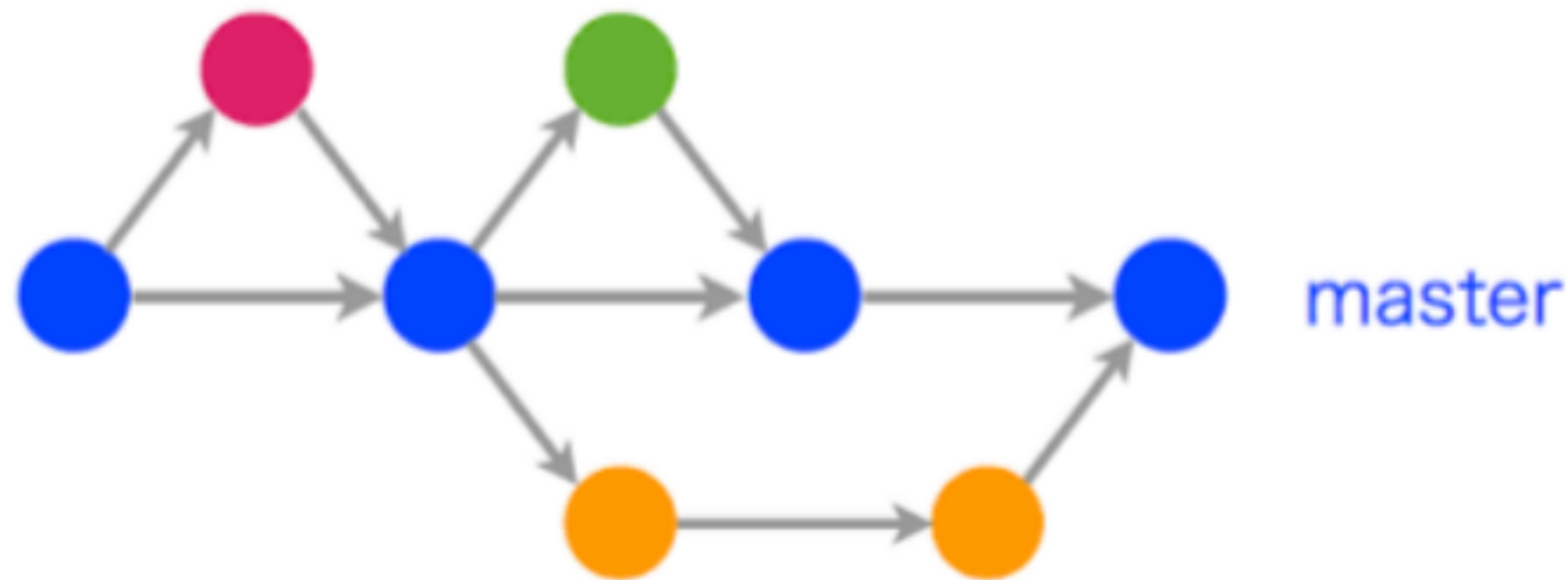
08 — 부록

- GUI 프로그램 소개
- 고급 명령어 소개

09 — 설치와 회원가입

- git
- visual studio code
- github

05. 브랜치



아주아주 중요한 프로그램을 개발한다면.
Main 브랜치를 어떻게 관리??

1. 일단 main에 넣어 배포해본다
2. 브랜치에서 검증을 해본 후 main에 배포한다

- 격리
- 협업(병렬적인 작업)

05. 브랜치



05. 브랜치

```
mai@fedora:~$ git status #현재 내 상황 알아보기
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

```
mai@fedora:~$ git branch feat/999 main #main에서 feat/999라는 이름의 브랜치를 생성
```

```
mai@fedora:~$ git switch feat/999 #feat/999라는 이름의 브랜치로 이동
Switched to branch 'feat/999'
```

```
mai@fedora:~$ git status #제대로 이동했는지 확인
On branch feat/999
nothing to commit, working tree clean
```

CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

08 — 부록

- GUI 프로그램 소개
- 고급 명령어 소개

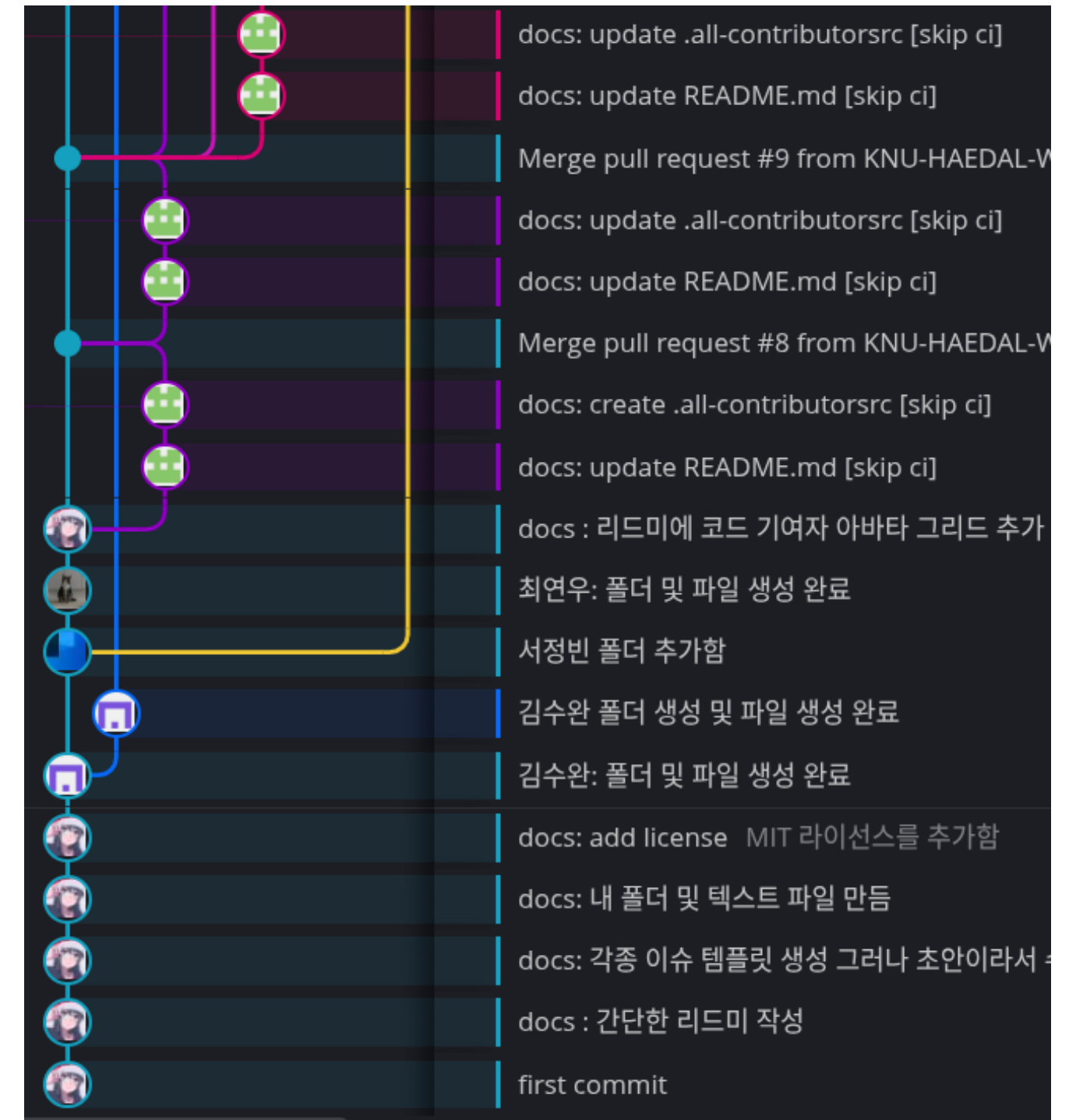
09 — 설치와 회원가입

- git
- visual studio code
- github

06. 버전관리와 세이프포인트



==



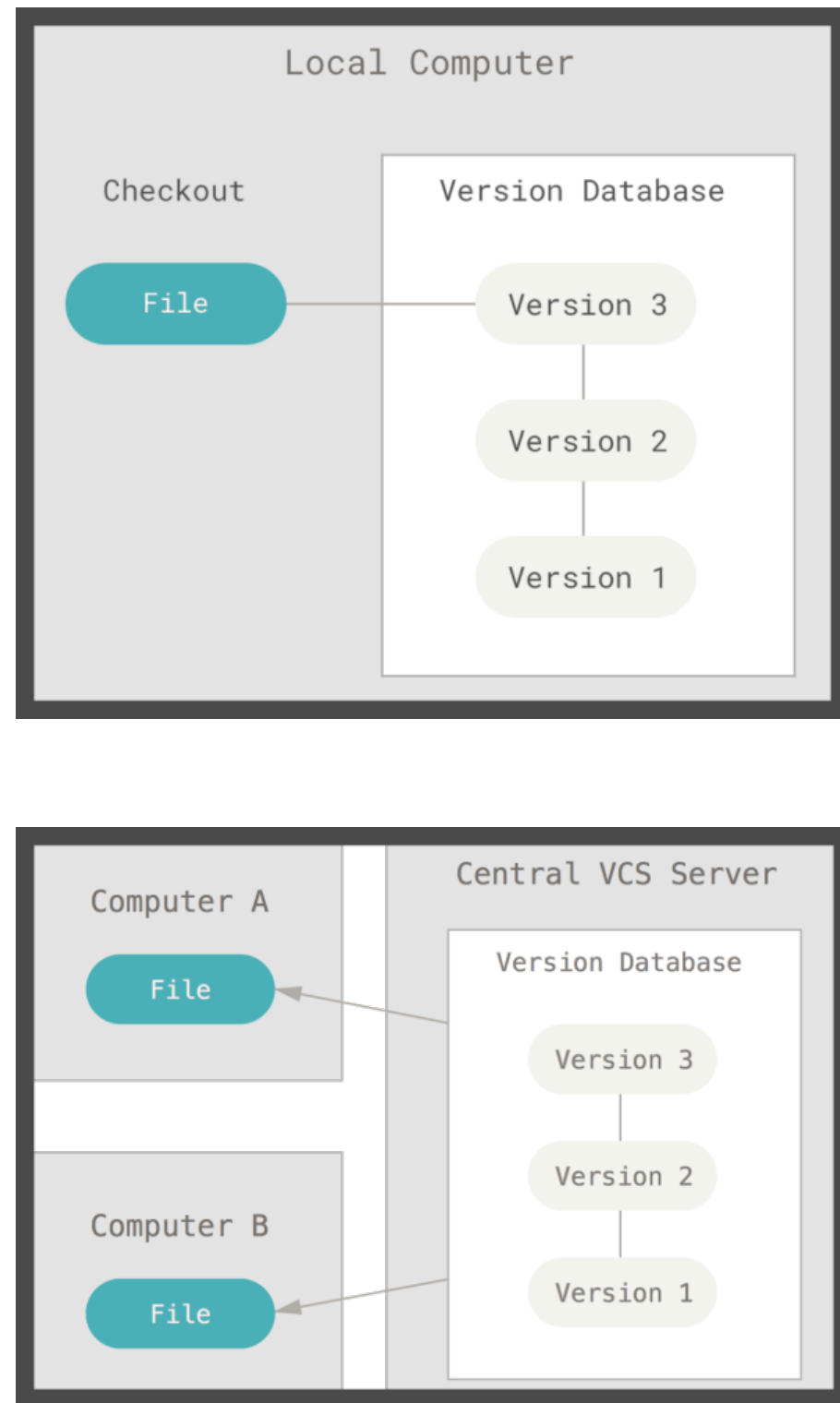
06. 버전관리와 세이프포인트

git add : ???

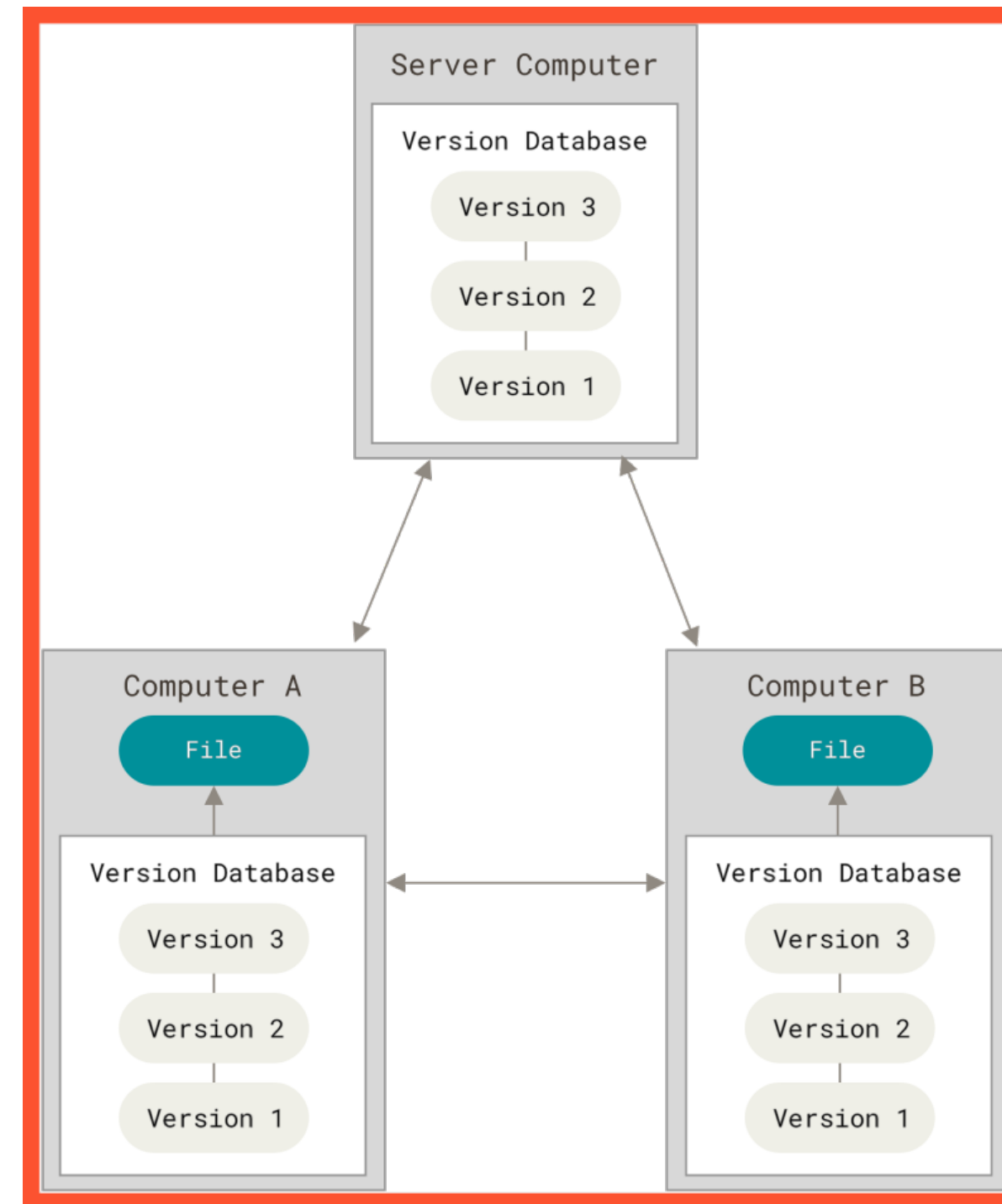
git commit : 세이프 포인트(스냅샷) 만들기

git push : 로컬 저장소의 변경 이력을 원격 저장소에 반영

01. Git이란?



분산 버전 관리 시스템(DVCS)



06. 버전관리와 세이프포인트

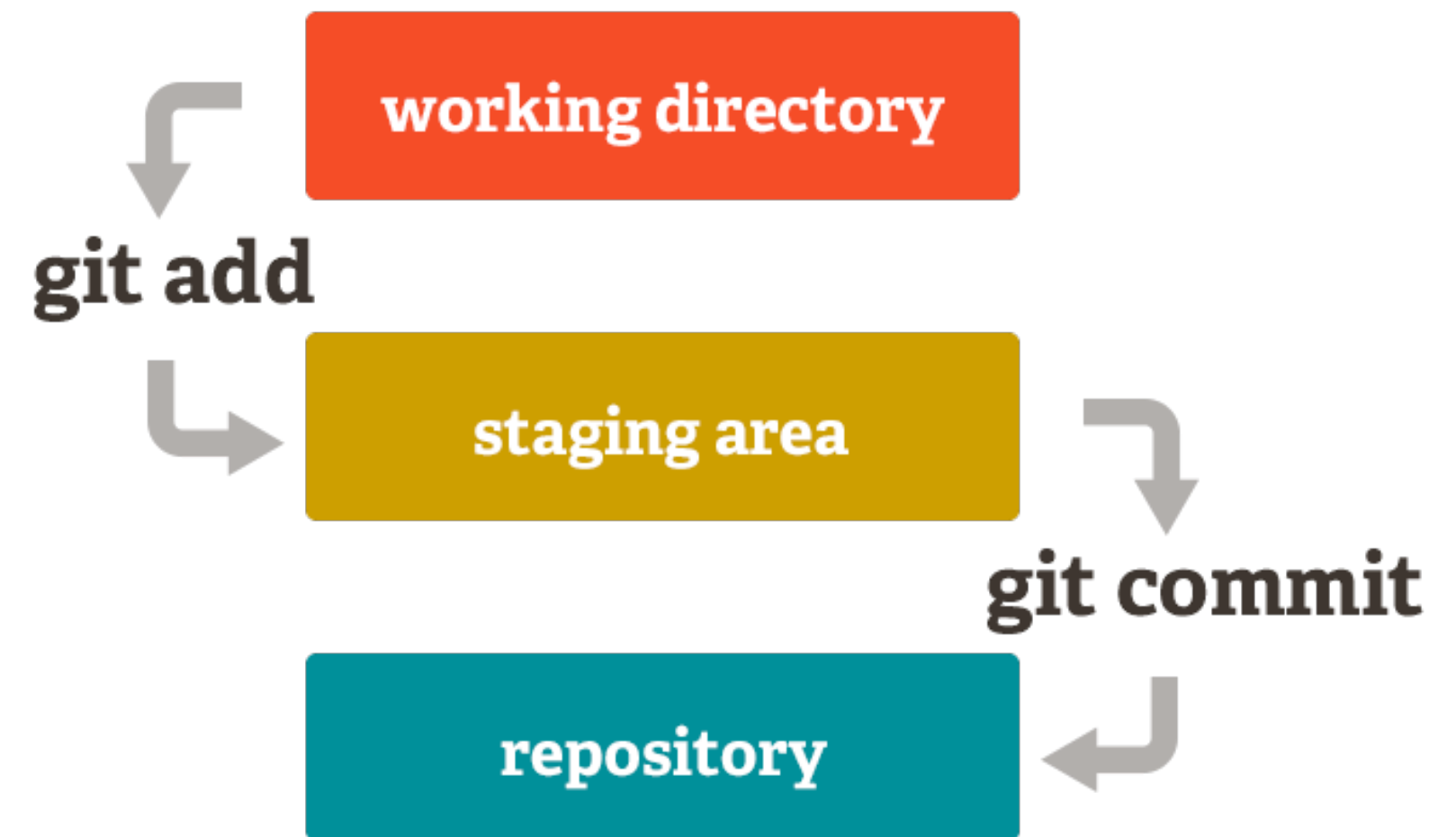
하나의 commit 에는 하나의 의미 있는 변경을 담는 것이 좋다.

06. 버전관리와 세이프포인트

```
✓ data
  TS bootcamp.data.ts
  TS contributors.data.ts
  TS highlight.data.ts
  TS join.data.ts
  TS projects.data.ts
  TS schedule.data.ts
  TS stats.data.ts
  TS tracks.data.ts
> layouts
✓ pages / Home
  ✎ Activities.tsx
  ✎ Contributors.tsx
  ✎ Hero.tsx
  ✎ Highlights.tsx
  ✎ index.tsx
  ✎ Join.tsx
  ✎ Projects.tsx
  ✎ Schedule.tsx
  ✎ Stats.tsx
  ✎ Tracks.tsx
```

왜 commit만 쓰는게 아니라,

add / commit으로 나눠봤을까?



06. 버전관리와 세이프포인트

```
# 프로그래밍 작업을 했다 ~~~
```

```
# 모든 파일을 staging area로 옮기기
```

```
mai@fedora:~$ git add .
```

```
# staging area의 내용들을 커밋 -> 세이프포인트 생성
```

```
# 이때 커밋에 대한 메모를 적어놓기 -> 세이프포인트가 되려면 메모해놔야지;;
```

```
mai@fedora:~$ git commit -m "feat : 로그인 기능 추가"
```

```
# 커밋 내용을 온라인으로 전송
```

```
mai@fedora:~$ git push
```

```
# 맨 처음에는 이렇게 해야함
```

```
# 여러 원격 저장소가 있을 때, 누구를 쓸건지 지정하는 것임
```

```
mai@fedora:~$ git push -u origin main
```

CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

08 — 부록

- GUI 프로그램 소개
- 고급 명령어 소개

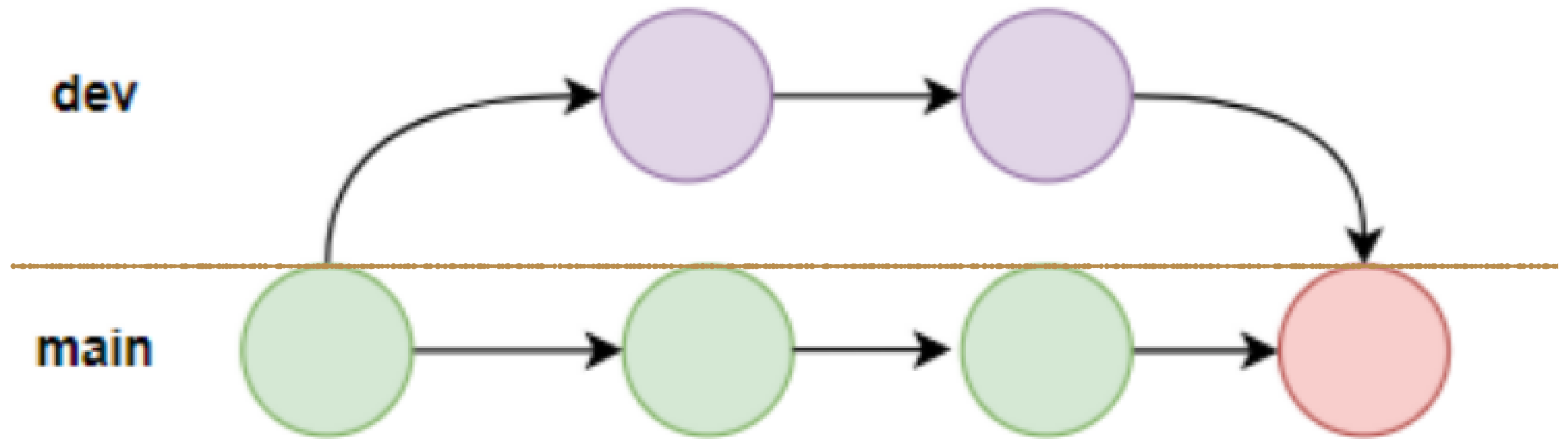
09 — 설치와 회원가입

- git
- visual studio code
- github

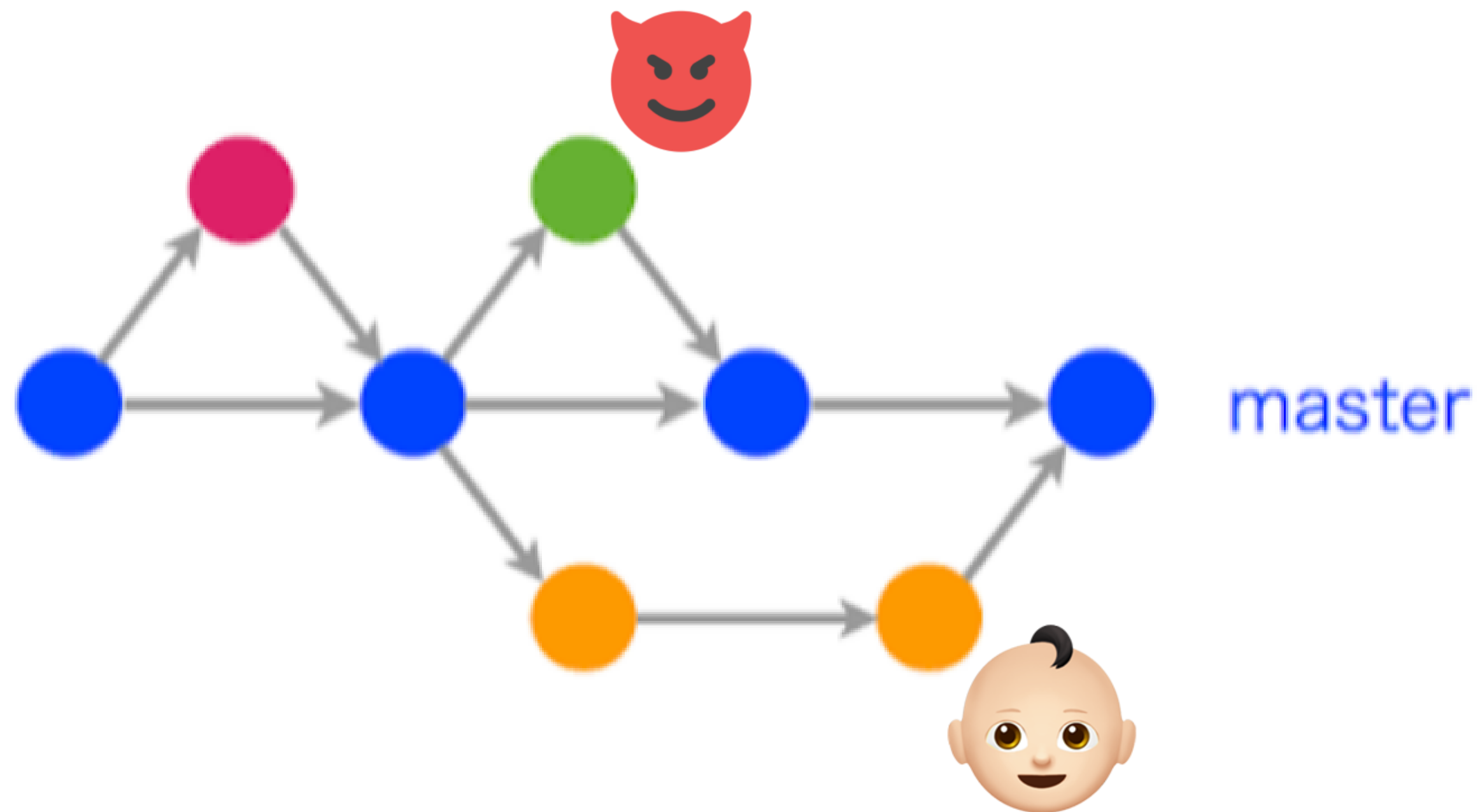
07. 병합과 병합요청

```
git merge 브랜치명
```

앞에서 branch, switch
add commit push 한거 다 여기임



07. 병합과 병합요청 -PR을 날리다



07. 병합과 병합요청 -PR을 날리다

feat : cors에러 수정 #97

Merged

dev-seonghyeon merged 2 commits into `main` from `dev`  on Jun 11

Conversation 0

Commits 2

Checks 1

Files changed 1

 All commits ▾

Filter files... 

backend/src/main/java/com/ag...

 CorsConfig.java

▼ backend/src/main/java/com/agent/harness/config/CorsConfig.java

↑...	@@ -23,7 +23,8 @@	public void addCorsMappings(CorsReg
23	23	"http://localhost:3000",
24	24	"http://localhost:5173",
25	25	"http://localhost:8080",
26	-	"http://localhost:80"
26	+	"http://localhost:80",
27	+	"https://app.haedal-sos-man-3
27	28	)
28	29	.allowedMethods("GET", "POST", "PUT",
29	30	.allowedHeaders("*")

.....

07. 병합과 병합요청 -conflict

```
best_food.md
1      1      2      3
      Accept current change | Accept incoming change | Accept both changes
1 <<<<<< feature/44-add-food (Current change)
2 #곽상원
3 {
4 강남오뎅, 오뎅이랑 물떡이 무한리필임.
5 사장님이 너무 착하시고 서비스 많이주심.
6 }
7 =====
8 동아리분식, 기숙사 근처, 제육볶음, 7000원- 된장이 맛있어요
9 김수완: 가게이름_효목, 위치_동성로, 저는 개인적으로 핫케(임연수) 솔밥 추천드립니다,
10
11 감래등, 내집근처, 야끼우동, 9500
12 굿모닝베이커리, 동대구신시장, 우유식빵, 2500?
13
14
15 >>>>>> main (Incoming change)
16
```

**충돌은 에러가 아니다
질문 하는 것.**

07. 병합과 병합요청 -최신화 하기

```
git fetch
```

```
git pull
```

(git fetch + git merge)

택배 비유

- fetch: 택배가 집 앞에 도착했습니다. 아직 상자는 안 뜯었어요. 내 방은 그대로입니다.
- pull: 택배를 받아서 바로 뜯어 방에 정리까지 합니다. 이때 기존에 있던 물건과 자리가 겹치면 → 충돌.

CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

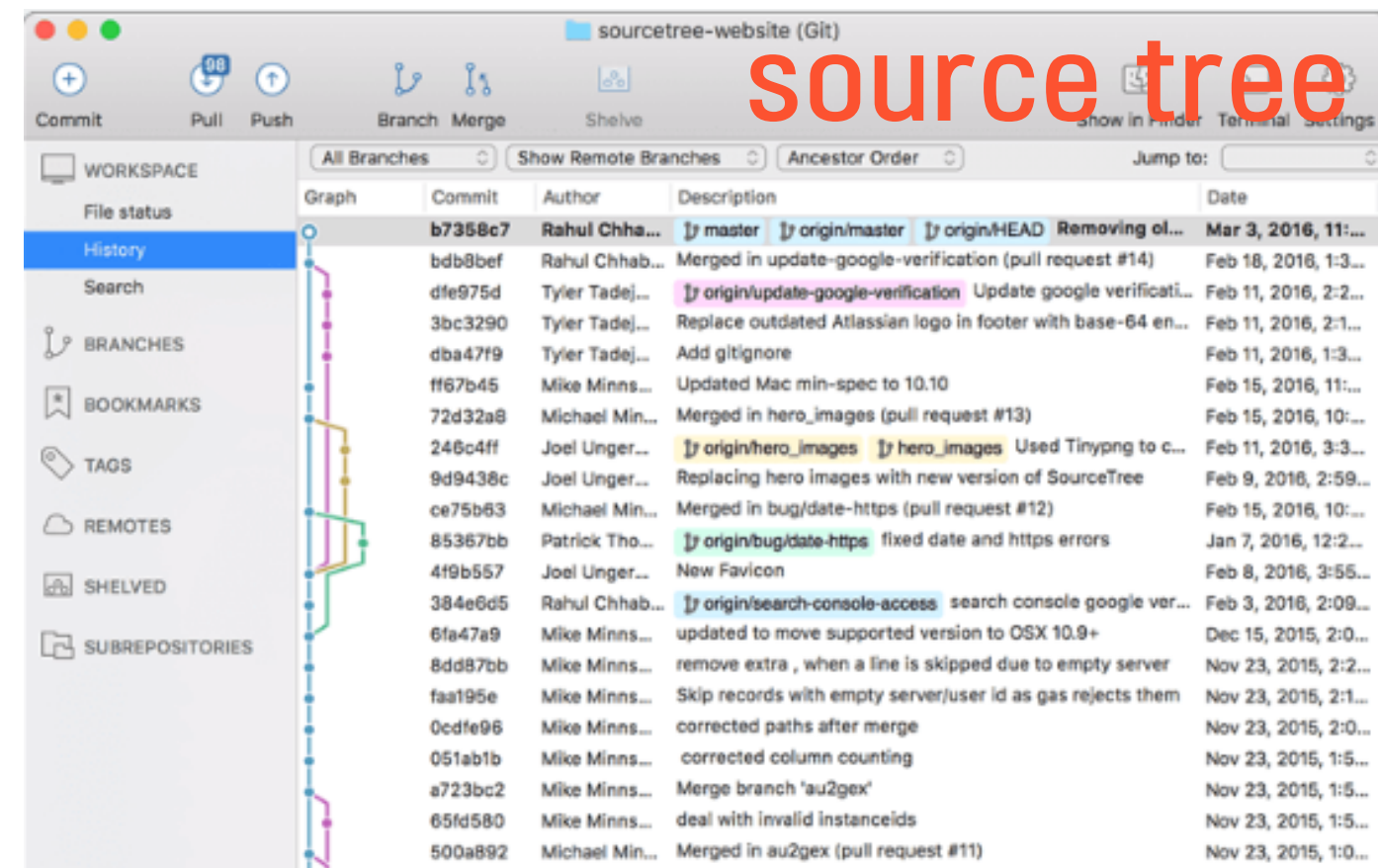
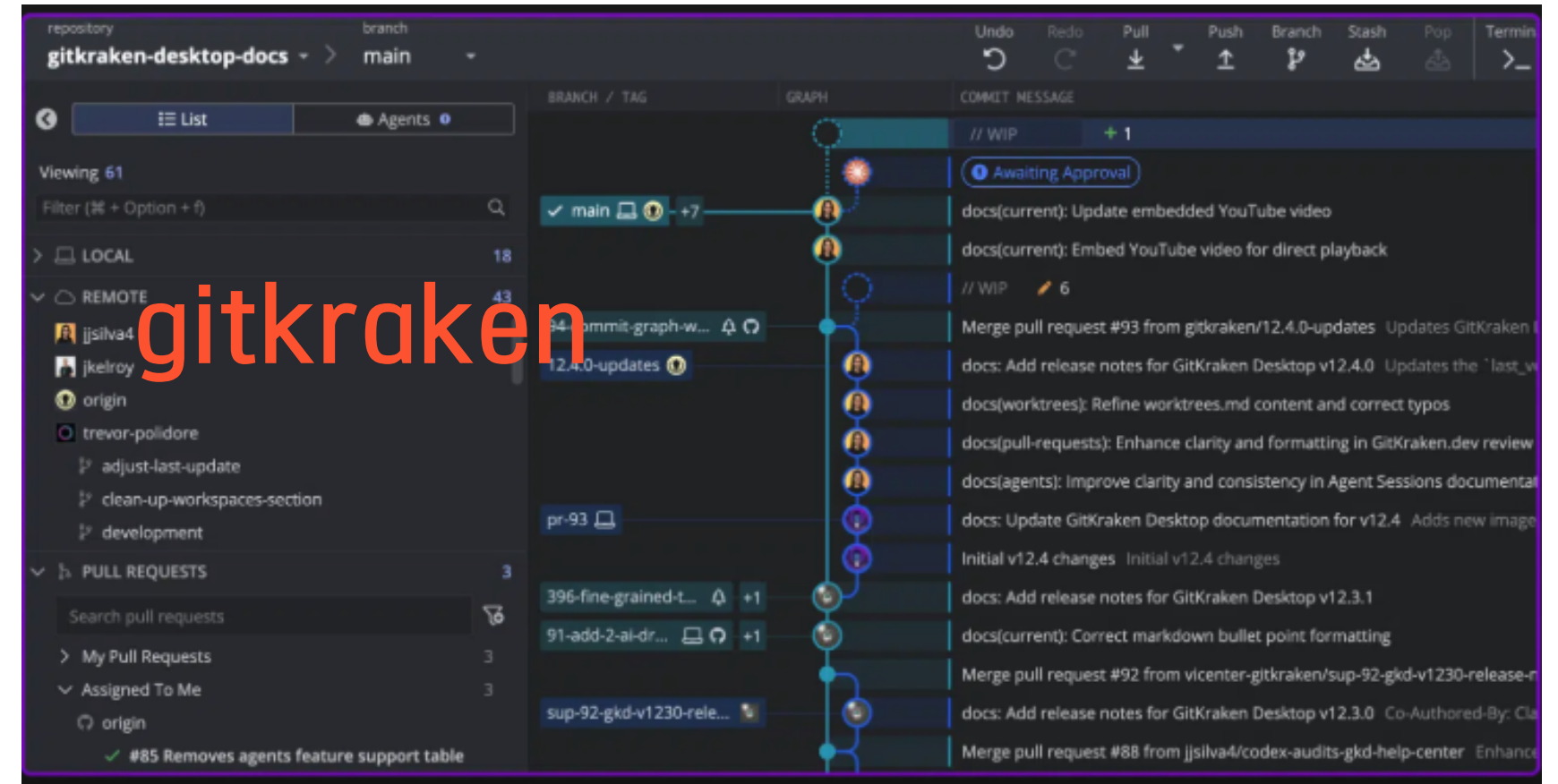
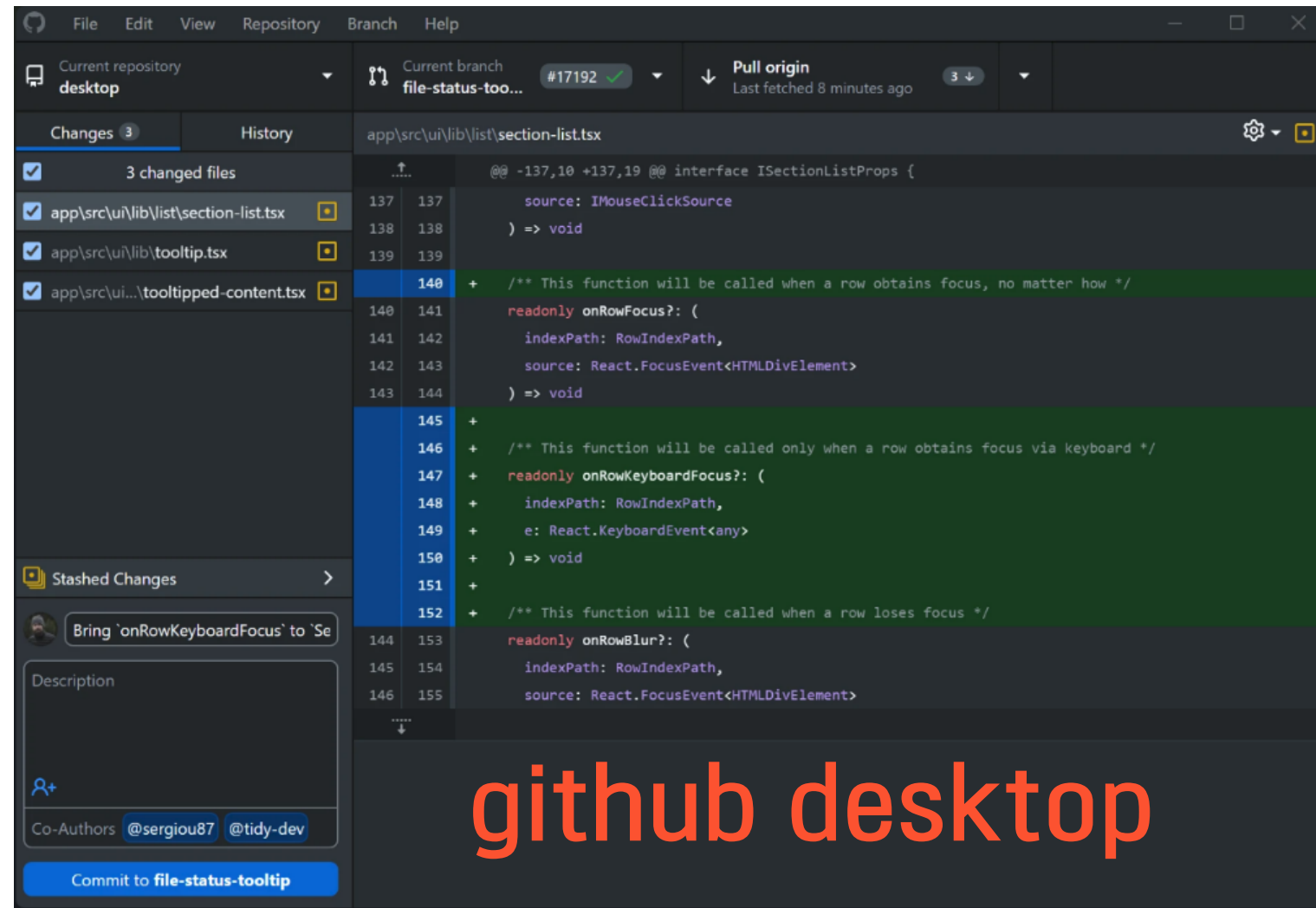
08 — 부록

- GUI 프로그램 소개
- 고급 명령어 소개

09 — 설치와 회원가입

- git
- visual studio code
- github

08. 부록 - GUI 프로그램



굉장히 편함!!

08. 부록 -고급 명령어

강의에서 “중급·고급 Git 명령어 – 이런 것도 있다” 정도로 소개한다면 아래 정도가 좋아.

- `git commit --amend` – 마지막 커밋의 내용이나 커밋 메시지를 수정한다.
- `git reset` – 브랜치가 가리키는 커밋 위치를 이전 상태로 되돌린다. 옵션에 따라 스테이징 영역이나 작업 파일도 함께 되돌릴 수 있다.
- `git revert` – 기존 커밋의 변경을 취소하는 새로운 커밋을 만든다. 이미 공유된 이력을 되돌릴 때 주로 사용한다.
- `git push --force` – 원격 브랜치의 이력을 로컬 이력으로 강제로 덮어쓴다. 협업 중에는 매우 주의해야 한다.
- `git push --force-with-lease` – force push 전에 원격 브랜치가 예상한 상태인지 확인한 뒤 강제로 push한다. 일반 `--force`보다 안전하다.
- `git rebase` – 커밋들의 기반(base)을 다른 커밋으로 옮겨 이력을 재구성한다.
- `git rebase -i` – 여러 커밋을 합치거나, 순서를 바꾸거나, 메시지를 수정하는 등 이력을 편집한다.

08. 부록 -고급 명령어

- `git reflog` - `HEAD`와 브랜치가 과거에 어디를 가리켰는지 로컬 기록을 확인한다. 잘못된 `reset`이나 `rebase` 복구에 특히 유용하다.
- `git cherry-pick <commit>` - 특정 커밋 하나의 변경사항만 현재 브랜치에 적용한다.
- `git stash` - 아직 커밋하지 않은 변경사항을 임시로 치워두고 나중에 다시 꺼낸다.
- `git bisect` - 정상 커밋과 문제가 생긴 커밋 사이를 이진 탐색하여 버그가 처음 발생한 커밋을 찾는다.
- `git restore` - 작업 트리나 스테이징 영역의 파일 변경을 복원하거나 취소한다.
- `git clean` - Git이 추적하지 않는 `untracked` 파일을 삭제한다. `-n`으로 먼저 확인하는 습관이 중요하다.
- `git merge --abort` - 충돌 중인 `merge`를 취소하고 `merge` 이전 상태로 돌아간다.
- `git rebase --abort` - 진행 중인 `rebase`를 취소하고 시작 전 상태로 돌아간다.

08. 부록 -참고할 자료들

[git 공식 문서 사이트](#)

[git cheatsheet](#)

[한국어 버전 cheatsheet](#)

[한국어 버전 cheatsheet2](#)

CONTENTS

01 — 인트로

- Git이란?

02 — 터미널과 CLI

- CLI란?
- 명령어의 구조
- 좋은 습관

03 — 초기 세팅

- config

04 — 레포지토리 세팅

- .git 폴더
- clone
- init

05 — 브랜치

- branch

06 — 버전 관리와 세이프 포인트

- add
- commit
- push

07 — 병합과 병합 요청

- PR
- conflict
- merge

08 — 부록

- GUI 프로그램 소개
- 고급 명령어 소개

09 — 설치와 회원가입

- git
- visual studio code
- github

09. 설치와 회원가입

[Git 설치방법 링크](#)

[Visual Studio Code 설치 링크](#)

[Github 링크](#)

*Github 회원가입이 잘 안되면 시크릿 창에서 하기
**참고로, Github 로봇이 아닙니다 인증이 정말 어렵습니다...



경청해주셔서
감사합니다